La Programmation Orientée Objet

Un peu de contexte

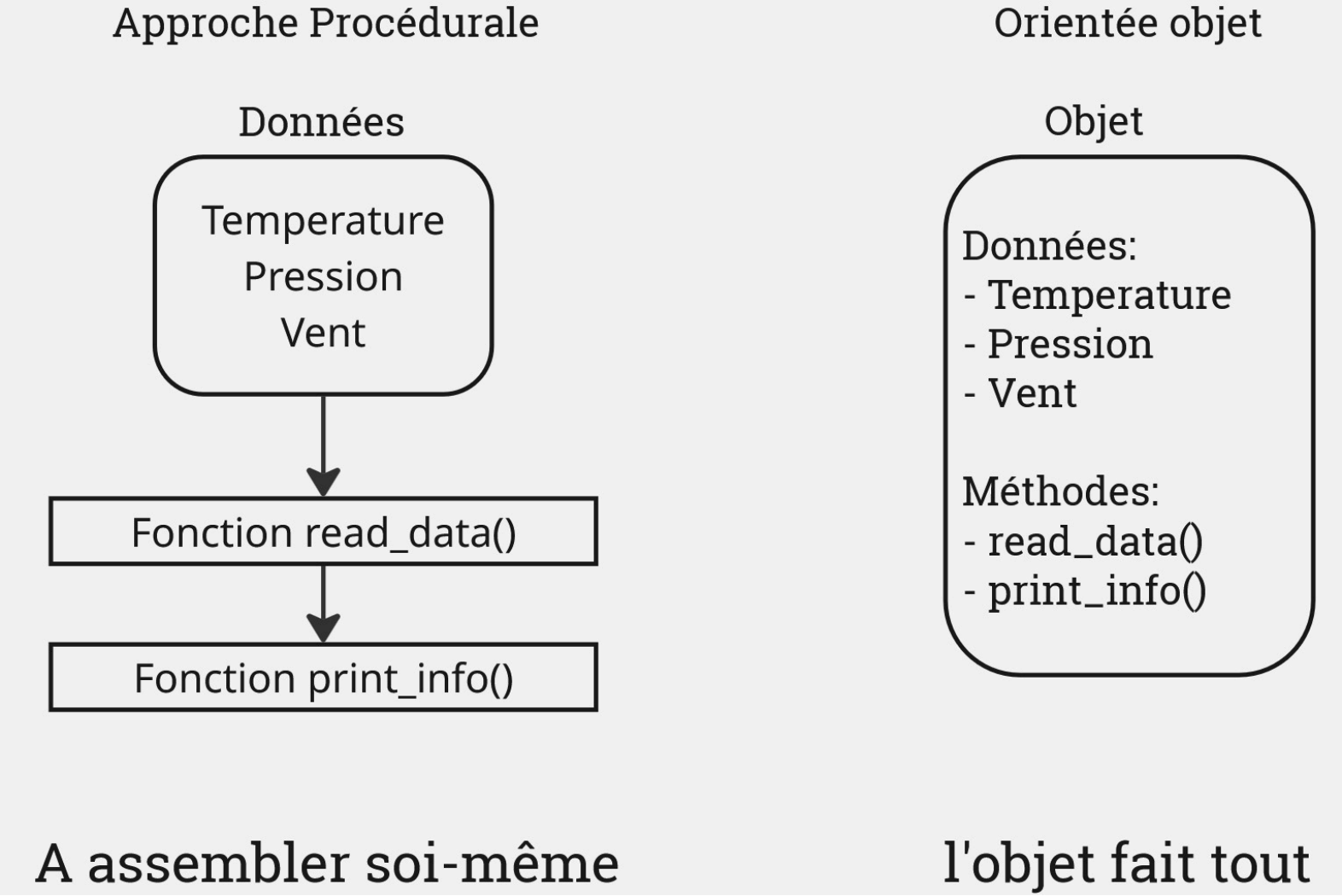
Dans le développement logiciel scientifique, beaucoup de programmeurs n'ont pas une formation poussée en informatique. Ils sont surtout experts en modélisation physique, méthodes numériques, maths appliquées et optimisation des performances.
Par conséquent, ils utilisent souvent une approche procédurale, qu'ils connaissent bien.

Mais dès qu'on parle de programmation orientée objet, les questions arrivent :

- Faut-il vraiment utiliser des objets ?
- Quels avantages par rapport au procédural ?
- Comment les mettre en place efficacement ?
- Quels sont les pièges à éviter ?

Et comme souvent… la réponse dépend du contexte : il n'y a pas de solution unique.

Procédurale vs Orientée objet



Lors du précédent TP nous avons défini plusieurs fonctions pour venir lire, filtrer et interpréter les données d'un fichier CSV relatif à un réseau de stations météo.

`read_station_data()` → lecture du fichier d'entrée et sélection d'une station via son ID.
`print_station_info()` → afficher des informations relatives à cette station.
`select_period()` → sélectionner une plage temporelle sur les données de la station.
`extrema()` → calculer pour une variable donnée la première heure de ses extrema.
`aggregation()` → calculer pour une variable donnée la moyenne sur chaque heure de la journée.
`visualize()` → visualiser des données/résultats via un graphique.

Références pour les fonctions:

```python
def read_station_data(station_id: int):
    """Returns a filtered dataframe with data from the required station only."""
    file_path = "/home/newton/horsenm/destouchesm/COURS_CS/data/station_2018.csv"
    df = pd.read_csv(file_path, parse_dates=[4])
    # Check that the id does exist.
    if station_id not in df["number_sta"].unique():
        print(f"La station demandée {station_id} n'existe pas.")
        print(f"Les possibilités sont {df['number_sta'].unique()}")
        raise ValueError(f"Station {station_id} does not exist!")
    # Filter rows based on station number
    return df[df["number_sta"] == station_id]

def print_station_info(station_id: int):
    """Find and print information on the location of a given station."""
    print(f" Information pour la station {station_id}")
    print(f" Latitude de la station : {data['lat'].unique()}")
    print(f" Longitude de la station : {data['lon'].unique()}")
    print(f" Hauteur de la station : {data['height_sta'].unique()}")

def select_period(
    df: pd.DataFrame,
    start_period: dt.datetime,
    end_period: dt.datetime,
    hour: int | None = None,
):
    """
    Returns a filtered dataframe with dates between `start_period` and `end_period`.
    If `hour` is provided, adds an additional filtering to return only data at
    this hour of the day.
    """
    condition = (df.date >= start_period) * (df.date <= end_period)
    # Update condition to add hour selection
    if hour is not None:  # Caution! `if hour:` would not work (when hour==0)
        condition = condition * (df.date.dt.hour == hour)
    df_period = df[condition]
    return df_period

def extrema(df: pd.DataFrame, variable: str):
    """
    For a given variable, return the first hour at which the maximum and minimum
    have been reached.
    """
    maxi = df[variable].max()
    mini = df[variable].min()
    # Filter to keep only elements where maximum has been reached.
    max_date = df[df[variable] == maxi]
    # Same for minimum.
    min_date = df[df[variable] == mini]
    max_hour = max_date["date"].dt.strftime("%H").values[0]
    min_hour = min_date["date"].dt.strftime("%H").values[0]
    return (max_hour, min_hour)

def aggregation(df: pd.DataFrame, variable: str, method: str = "mean"):
    """
    Return mean, min or max the required variable, depending on the chosen
    aggregation method.
    """
    result = []
    for hour in range(24):
        cdt = df.date.dt.hour == hour
        df_selected = df[cdt]
        if method == "mean":
            result.append(df_selected[variable].mean())
        elif method == "min":
            result.append(df_selected[variable].min())
        elif method == "max":
            result.append(df_selected[variable].max())
        else:
            raise ValueError("Aggregation method not known")
    return result

def visualize(x_value, y_value, axis_labels=["X", "Y"]):
    """Plot the y vs x values on a graph with the possibility of specified axis labels"""
    plt.plot(x_value, y_value)
    plt.xlabel(axis_labels[0])
    plt.ylabel(axis_labels[1])
    plt.show()
```

Comment un objet peut apparaître

Les objets sont des outils utiles pour simplifier l'interface de programmation (API).

Prenons notre script principal, notre API, qui ressemble pour l'instant au suivant:

```
# Script d'appel pour la station 73010
station_id = 73010
df_station_73010 = read_station_data(station_id)
print_station_info(df_station_73010, station_id)
df_station_73010_oct = select_period(df_station_73010, "2018-10-1", "2018-10-15")
h_temp_max, h_temp_min = extrema(df_station_73010_oct, "t")
mean_temp = aggregation(df_station_73010_oct, "t", method="mean")
visualize(range(len(mean_temp)), mean_temp, axis_labels=["Heure de la journée", "Température Moyenne"])
```

Quels sont les défauts d'un tel script ?

```
# Script d'appel pour la station 73010
station_id = 73010
df_station_73010 = read_station_data(station_id)
print_station_info(df_station_73010, station_id)
df_station_73010_oct = select_period(df_station_73010, "2018-10-1", "2018-10-15")
h_temp_max, h_temp_min = extrema(df_station_73010_oct, "t")
mean_temp = aggregation(df_station_73010_oct, "t", method="mean")
visualize(range(len(mean_temp)), mean_temp, axis_labels=["Heure", "Température Moyenne"])
```

- Répétition des arguments (id, df) à chaque appel.
- Pas de lien entre les fonctions → risque d'erreur et incohérence.
- Aucun état conservé → il faut tout repasser à chaque fois.
- Évolution difficile → une modification touche plusieurs fonctions.
- Code peu réutilisable → compliqué à étendre à plusieurs stations/périodes.

L'API idéale pourrait ressembler à cela:

```
station_73 = StationMeteo(station_id=73010)
station_73.set_period("2018-10-1", "2018-10-15")
h_max, h_min = station_73.extrema("t")
mean_T = station_73.aggregate('t', method='mean')
visualize(range(24), mean_T, axis_labels=["Heure", "Température moyenne"])
```

- `station_73` est un *objet* `StationMeteo()` défini pour la station 73010.
- Cela s'appelle une *instance* de l'objet `StationMeteo`.
- `.set_period()` est une *méthode* de l'objet qui remplace la fonction `select_period()`.
- Ce qui rend cet objet unique est son *attribut* `station_id`.

Comment créer ce créateur d'objet qu'est `StationMeteo()` ?

On définit une *classe*:

```
class StationMeteo:
    def __init__(self, station_id: int):
        self.station_id = station_id
        self.df = read_station_data(self.station_id)
        self.df_period = self.df

    def set_period(self, start, end):
        self.df_period = select_period(self.df, start, end)

    def info(self):
        print_station_info(self.df_period, self.station_id)

    def extrema(self, var: str):
        return extrema(self.df_period, var)

    def aggregate(self, var: str, method: str):
        return aggregation(self.df_period, var, method=method)
```

Cette classe possède une méthode d'initialisation `__init__()` :

```
    def __init__(self, station_id):
        self.station_id = station_id
        self.df = read_station_data(self.station_id)
        self.df_period = self.df
```

Ainsi que plusieurs méthodes dont:

```
    def info(self):
        print_station_info(self.df_period, self.station_id)
```

Les méthodes sont comme des super fonctions qui, lorsqu'elles sont définies dans une classe, peuvent utiliser des attributs comme ici `self.station_id` qui sont des paramètres spécifiques de l'objet.

Exercice

- Mettre en place la classe `StationMeteo` dans le script et exécuter une procédure d'appel complète avec instanciation d'un objet `StationMeteo` pour la station 22219003 et la visualisation de la température moyenne pour chaque heure sur la période du 1er au 5 février 2018.
- Même demande pour la température maximale sur la période estivale.
- Ajouter la possibilité d'exporter sous forme de .csv les données de l'objet station, avec l'option de sauver la période en cours d'étude seulement.

Solution potentielle

```
station_22 = StationMeteo(22219003)
# Calcul et visu température moyenne 1 au 5 Février
station_22.set_period("2018-2-1", "2018-2-5")
mean_T = station_22.aggregate("t", method="mean")
visualize(range(24), mean_T, axis_labels=["Heure", "Température moyenne"])

# Calcul et visu température maximale période estivale
station_22.set_period("2018-6-1", "2018-8-31")
max_T = station_22.aggregate("t", method="max")
visualize(range(24), mean_T, axis_labels=["Heure", "Température moyenne"])
```

Méthode de la classe StationMeteo pour l'export:

```
def export(self, period=False):
        if not period:
            self.df.to_csv(f"station_{self.station_id}.csv")
        else:
            self.df_period.to_csv(f"station_{self.station_id}.csv")
```

Vers un objet Reseau

Bien que notre objet `StationMeteo` réponde désormais à nos besoins, traiter simultanément toutes les stations du fichier aboutirait à un code complexe, difficile à maintenir et susceptible d'introduire des erreurs, à l'image de l'approche fonctionnelle utilisée au départ.

Exercice

- Créer un objet/classe `ReseauMeteo` permettant d'avoir accès à n'importe quelle station. L'utilisation de l'objet `StationMeteo` est recommandé...
- Mettre en place une méthode permettant d'afficher les informations des différentes stations du réseau.
- Bonus : Ajouter la possibilité de filtrer la période sur tout le réseau

```python
class Reseau:
    def __init__(self, file_path: str):
        self.file_path = file_path
        self.stations = {}
        self._load_stations()

    def _load_stations(self):
        df = pd.read_csv(self.file_path, parse_dates=[4])
        for station_id in df["number_sta"].unique():
            self.stations[station_id] = Station(station_id, station_df)

    def get_station(self, station_id: int):
        return self.stations.get(station_id)

    def info(self):
        for id in self.stations:
            print(10 * "-")
            self.stations[id].info()

    def set_period(self, start, end):
        for station in self.stations.values():
            station.set_period(start, end)
```

On effectue ici ce que l'on appelle une *composition d'objets*: un objet composé d'autres objets.

Cela permet de se retrouver avec l'API suivante:

```
res = Reseau("../station_2018.csv")
res.info()

res.set_period("2018-2-1", "2018-2-5")
mean_T_A = res.get_station(28206001).aggregate("t", method="mean")
mean_T_B = res.get_station(85191003).aggregate("t", method="mean")
visualize(
    range(24),
    abs(np.array(mean_T_A) - np.array(mean_T_B)),
    axis_labels=["Heure", "Température moyenne"],
)
```

Concept d'API progressive

Nom	Type	Niveau	Situation
`Reseau()`	Composed object	Haut	Exploiter un réseau de stations
`StationMeteo()`	dedicated object	Moyen	Exploiter une seule station
`aggregation()`	dedicated function	Bas	Calculer une donnée statistique
`print_station_info()`	atomic function	Très bas	Afficher des infos
`read_data()`	atomic function	Très bas	Lire un fichier csv

Cette API est « progressive » : les nouveaux utilisateurs peuvent l'utiliser à un niveau élevé, tandis que les utilisateurs plus avancés, disposant d'une meilleure compréhension, ont accès à des fonctions de plus bas niveau.
Cette liberté ne nécessite pas de duplication de code. En effet, en relisant le code source, on constate que chaque nouveau niveau est construit par-dessus le précédent.

Ce type d'API progressive est fondamental dans la programmation orientée objet et en est par conséquent un de ses atouts majeurs.